

АДАПТИВНЫЙ ПОДХОД К ТОНКОЙ НАСТРОЙКЕ ЯЗЫКОВЫХ МОДЕЛЕЙ ДЛЯ ЗАДАЧ ПРЕОБРАЗОВАНИЯ ТЕКСТА В SQL

© 2025 Босенко Т.М.

Возможность генерации SQL-запросов на основе естественного языка существенно упрощает доступ к данным для пользователей, не обладающих техническими навыками. Современные методы часто полагаются на большие языковые модели (LLM) с закрытым исходным кодом, такие как ChatGPT, которые имеют ограничения, включая высокую стоимость и непрозрачную архитектуру. В данной работе предложен адаптивный подход к тонкой настройке малых языковых моделей (SLM) для преобразования текста в запросы SQL (Text-to-SQL), адаптированных под диалекты SQLite3 и PostgreSQL. Предложенная модель SQL_DS_100M_ft основывается на открытой архитектуре CodeS, что обеспечивает ее гибкость и доступность. Проведённый анализ показал, что тонкая настройка позволяет значительно улучшить точность генерации SQL-запросов (до 72.2% по метрике EX на тестах BIRD) и адаптивность модели для сложных аналитических задач. Основные преимущества таких моделей заключаются в структурированности генерируемых запросов, снижение затрат на вычислительные ресурсы по сравнению с LLM и открывают новые перспективы по улучшению доступности реляционных баз данных через естественно-языковые интерфейсы.

Ключевые слова: Text-to-SQL, языковая модель, SQL-запросы, контролируемая тонкая настройка, CodeS.

Введение. Языковые модели показали многообещающую производительность при выполнении задачи перевода вопросов на естественном языке в запросы SQL (текст в SQL). Тем не менее, большинство современных методологий опираются на высокопроизводительные, однако проприетарные большие языковые модели (LLM), такие как ChatGPT и GPT-4. Эти модели характеризуются неопределённой архитектурой, потенциальными рисками для конфиденциальности данных, а также значительными затратами на обеспечение вычислительной инфраструктуры, необходимой для поддержки многопользовательского доступа к системе [1]. Задача преобразования текста в SQL включает в себя перевод вопроса пользователя на естественном языке (NL) в соответствующий и допустимый запрос на языке структурированного запроса (SQL), который может быть выполнен с использованием запроса в соответствующую базу данных. Чтобы устранить перечисленные ограничения, был проведена тонкая настройка малой языковой модели (SLM) SQL_DS_100M_ft.

SQL_DS_100M_ft – это языковая модель размером в 100 миллионов параметров, предназначенная для решения задач Text-to-SQL (генерация SQL-запросов из текста) и прошедшая этап тонкой настройки (Fine-Tuning) адаптирована для использования в контексте ограниченного набора данных в определённой предметной области (DS – Data Science или Database System) на основе CodeS [2].

Эта модель предназначена для преобразования текста на естественном языке в SQL-запросы, обучена на синтетических и реальных наборах данных, адаптированных под диалекты SQLite3 и PostgreSQL, предназначена для работы с SQL-запросами. CodeS – это полностью открытая исходная языковая модель, которая достигает превосходной точности при гораздо меньших размерах параметров. По сравнению с решениями LLM text-to-SQL, CodeS имеет следующие преимущества:

- Полностью открытый исходный код LLM. CodeS, созданный на основе StarCoder [3], полностью открыт и доступен пользователям.
- CodeS достигает производительности сопоставимой с LLM почти во всех сложных тестах преобразования текста в SQL, таких как Spider [4] и BIRD [5].
- Малые размеры. CodeS в 10-100 раз меньше существующих LLM, таких как ChatGPT и GPT-4.

В этой работе представлен адаптивный подход к тонкой настройке языковой модели преобразования текста в SQL, адаптированной к определенному контексту. Обучение модели выполнялось на заранее размеченных данных, готовых шаблонах баз данных для продуктовой аналитики с использованием PostgreSQL и SQLite 3.

Задача преобразования текста в SQL. Цель преобразования текста в SQL – сгенерировать запрос SQL (S) на основе вопроса на естественном языке Q и базы данных D , чтобы запрос SQL можно было выполнить для ответа на вопрос:

$$S = \text{Parser}(Q, D),$$

где $S = \text{Parser}()$ предназначен для интерпретации предоставленного Q с помощью D и создания S . D содержит схему базы данных (т. е. таблицы и столбцы) и метаданные базы данных, которые содержат типы столбцов, комментарии, значения столбцов, первичные ключи и связи внешнего ключа.

Пусть дан набор из N языковых моделей $\{M_1, M_2, \dots, M_{N-1}\}$ и вопрос на естественном языке Q . Требуется определить самую слабую модель M_i , которая может точно сгенерировать соответствующий $\text{Parser}(Q, D)$ (S) для Q . Слабой моделью называют модель с низкой способностью генерации SQL-запросов.

Входные данные для Text-to-SQL – это приглашение, которое включает в себя вопрос на естественном языке (NL), сведения о базе данных и соответствующий диалект SQL. Выходные данные – это запрос SQL, который отвечает на поставленный вопрос. Информация о базе данных включает в себя несколько таблиц, каждая из которых имеет имя, столбцы и типы данных для столбцов.

Предварительно обученные языковые модели. Языковые модели, в первую очередь основанные на архитектуре Transformer [6], отлично справляются с задачами понимания и генерации текста. Обычно они проходят начальную фазу предварительного обучения на обширных текстовых данных с использованием целей неконтролируемого обучения (без учителя). Две известные парадигмы неконтролируемого обучения – это «моделирование языка» и «моделирование замаскированного языка». В первом случае такие модели, как GPT [7], LLaMa [8], предсказывают следующее слово из заданного контекста. В моделировании замаскированного языка определенные слова или словосочетания во входных данных случайным образом маскируются, и задача состоит в том, чтобы реконструировать замаскированные сегменты на основе немаскированного контекста. Представителями таких моделей являются предварительно обученные языковые модели, например, BERT [9] и T5 [10].

Контролируемая тонкая настройка (обучение с учителем) и контекстное обучение. Предварительно обученные языковые модели обладают обширными лингвистическими знаниями, однако для решения конкретных задач часто требуются уникальные языковые шаблоны или экспертные знания в предметной области. Для решения такого типа задач, контролируемая тонкая настройка (КТН) включает в себя профильное дообучение языковой модели на маркированных данных специализированной предметной области, при этом за основу берутся результат предварительного обучения, который получается из нового набора данных.

В отличие от KTH, концепция «обучения в контексте» [11] позволяет языковым моделям решать новые задачи, предоставляя соответствующие подсказки во входных данных, без необходимости в KTH. Однако эффективность обучения в контексте в значительной степени зависит от качества подсказок и самой языковой модели.

Разработанная SML SQL_DS_100M основывается на известных бенчмарках Text-to-SQL, таких как Spider, WikiSQL, CoSQL и SParC. Эти наборы данных предоставляют стандартные схемы и соответствующие запросы SQL, которые используются для обучения модели.

SML SQL_DS_100M_ft дополнена набором данных из бенчмарка «Data Scientist БИ», который включает расширенные схемы данных, процедуры, функции и триггеры. Это обеспечивает её более высокую адаптивность к реальным сценариям, связанным с обработкой сложных аналитических запросов. Этот подход позволяет учитывать дополнительные пары данных (вопрос, SQL-ответ), что улучшает качество прогнозируемых запросов в определенной предметной области.

Алгоритм работы модели SQL_DS_100M_ft.

1. Используются наборы данных (Spider [12], WikiSQL[13], CoSQL[14], SParC[15]) для создания обучающей выборки для малой языковой модели SQL_DS_100M.

2. В модель SQL_DS_100M_ft добавляется бенчмарк «Data Scientist БИ», включающий данные реальных схем баз данных, процедуры, функции и триггеры, которые обогащают её тренировочные данные.

3. На первом этапе модели CodeS передаётся пара «Database prompt + вопрос», что позволяет преобразовать вопрос (Q) в SQL (S) через базовый процесс обработки.

4. Настройка параметров KTH CodeS происходит посредством реальных обучающих данных, что приводит к улучшению логического вывода и формированию точного SQL-запроса (S).

В контексте образовательной парадигмы Московского городского педагогического университета была разработана SLM языковая модель, на основе модели CodeS. Наличие аутентичных обучающих данных, структурированных в соответствии со схемами баз данных, позволило осуществить прямую адаптацию параметров модели CodeS, подгоняя их под распределение размеченных примеров, как проиллюстрировано на рисунке 1. Такой подход обеспечивает оптимизацию процесса обучения и повышает точность результатов, достигаемых с помощью модели CodeS.

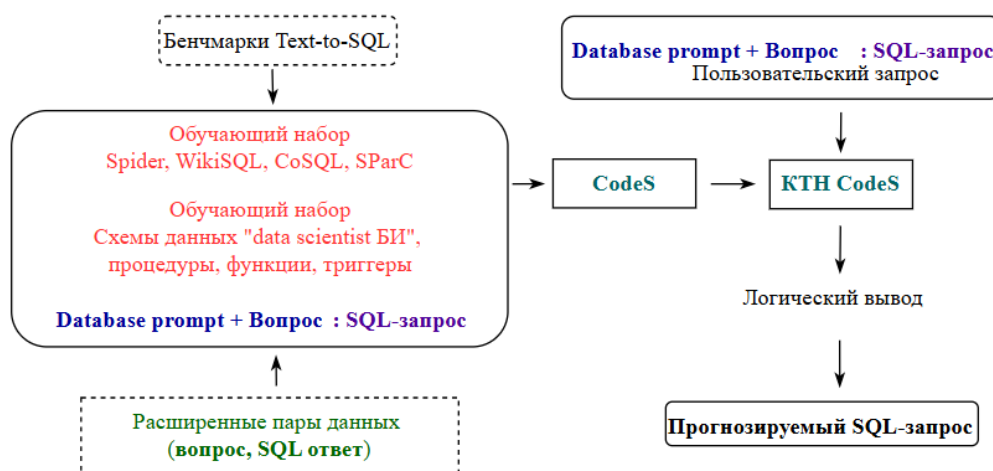


Рис. 1. Контролируемая тонкая настройка CodeS

Метрики оценки. Чтобы оценить эффективность сгенерированного SQL-ответа, используется в качестве основного показателя - точность выполнения (EX). В частности, $EX(S)$ обозначает долю запросов в оценочном наборе S , для которых выходное отношение (из прогнозируемого SQL-запроса) совпадает с отношением истинности. Следовательно, диапазон применимости EX : $0 \leq EX(S) \leq 1$.

Методология. Рассмотрим систему распределения запросов, которая основана на регрессионной модели, с целью оценки вероятности корректной генерации SQL-запроса конкретной моделью M_i для входного запроса Q и принятия решения об использовании модели или отказе от генерации запроса.

Этапы работы модели:

1. Пусть R_α^K – регрессионный маршрутизатор, где (α) – порог вероятности, K – количество похожих запросов, используемых для оценки. Принимает решение о выборе наиболее подходящей модели для обработки входного запроса (Q) на основе исторической информации о похожих запросах и вероятности успешного их выполнения, оптимизируя баланс между точностью, стоимостью и производительностью. Прогнозирует для каждой модели M_i вероятность $P(EX = 1_{M_i} | Q)$, которая определяет способность модели M_i сгенерировать корректный SQL для запроса Q .

2. Маршрутизатор использует порог вероятности (α) для фильтрации моделей. Модель M_i не может сгенерировать запрос, если $P(EX = 1_{M_i} | Q) < \alpha$.

3. Задача маршрутизатора – минимизировать индекс (i) такой, что $P(EX = 1_{M_i} | Q) \geq \alpha$, при этом он выбирает «самую слабую» модель, которая удовлетворяет пороговому значению (α) .

4. Если ни одна из моделей не достигает порогового значения вероятности (α) , то либо запрос не генерируется, либо он направляется к модели, которая обеспечит баланс между точностью и вычислительными затратами. В данной реализации для максимизации точности выбирается «самая сильная» модель.

Механизм оценки и маршрутизации:

1. Для данного запроса (Q) ищем (K) наиболее похожих запросов (Q_k) из исторических данных (H).

2. Для каждой модели (M) вероятность $P(EX = 1_{M_i} | Q)$ задаётся как $EX(Q_k | M_i)$, где $EX(Q_k | M_i)$ – метрика, которая показывает: как модель (M_i) справлялась с похожими запросами (Q_k).

3. Модели фильтруются относительно порогового значения (α) : остаются только те модели, для которых $EX(Q_k | M_i) \geq \alpha$.

4. Из моделей, которые прошли фильтр, выбирается «самая слабая» с минимальным значением $EX(Q_k | M_i)$, которая удовлетворяет пороговому значению (α) , для снижения вычислительной сложности.

Постановка задачи. Общая постановка задачи для выбора оптимальной модели SQL_DS_100M_ft для генерации SQL-запросов на основе обученной модели BERT:

максимизировать $EX(S)$, где S - SQL-запрос, сгенерированный выбранной моделью M_i , при условии, что M_i является самой слабой моделью, способной сгенерировать точный SQL-запрос. Формализация:

$R_{BERT}: (NL, D \text{ схема базы данных}) \rightarrow l$, где $l \in \{1, 2, \dots, n, N\}$

Если $l = i$, где $i \in \{1, 2, \dots, n\}$, то:

- $EX(Q) = 1_{M_i}$, если модель M_i генерирует точный SQL-запрос для Q
- $M_i = \operatorname{argmin}\{M_j \mid EX(Q) = 1_{M_j}, j \in \{1, 2, \dots, n\}\}$

Если $l = N$, то:

- $EX(Q) = 0$ для всех моделей M_i

Задача оптимизации для поиска языковой модели: максимизировать $EX(S)$ при условии:

- $S = \text{SQL-запрос, сгенерированный моделью } M_i$
- $M_i = \operatorname{argmin}\{M_j \mid EX(Q) = 1_{M_j}, j \in \{1, 2, \dots, n\}\}$

Обучение и оптимизация поиска модели выполнялись с использованием набора данных BIRD [16], который является сложным тестом для Text-to-SQL. Обучающий набор состоял из 1534 запросов в наборе данных, а обучающий набор «Data Scientist БИ», который выступал в качестве (H), состоял из 1890 запросов.

В таблице 1 указаны показатели точности выполнения ($EX\%$) для разных моделей при обработке запросов из набора данных BIRD. В таблице учтены модели различных архитектур, включая SQLCoder-7B-2, text2sql, WPAIGPT-sql-01, две модели SQL_DS_100M и две модели SQL_DS_100M_ft с различными порогами вероятности (α).

Языковая модель SQLCoder-7B-2 остается лидером по точности, но его применение может быть ограничено из-за высоких требований к вычислительным ресурсам.

Таблица 1. Распределение моделей при каждой генерации SQL-запроса

Модель	Архитектура	EX%
SQLCoder-7B-2 [17]	CodeLlama-7B	76.9
text2sql [18]	Google T5 Base	69.8
WPAIGPT-sql-01[19]	Qwen2.5-Coder-7B-Instruct	66.5
$R_{0.6}^{25}(\text{SQL_DS_100M})$ [2]	CodeS	64.8
$R_{0.7}^{38}(\text{SQL_DS_100M})$	CodeS	66.1
$R_{0.6}^{15}(\text{SQL_DS_100M_ft})$	CodeS	71.8
$R_{0.7}^{15}(\text{SQL_DS_100M_ft})$	CodeS	72.2

SLM $R_{0.7}^{15}(\text{SQL_DS_100M_ft})$ представляет собой оптимальный выбор для практических приложений, предлагая высокую точность (72.2%) при значительно меньших вычислительных затратах. Это особенно важно для образовательных целей и анализа данных в экономических и технических областях, где ресурсы могут быть ограничены. Архитектура CodeS в сочетании с тонкой настройкой ($_ft$) демонстрирует конкурентоспособные результаты, что указывает на потенциал данной архитектуры для дальнейшего развития и применения в задачах Text-to-SQL.

Результаты. Рассмотрим, как использование тонкой настройки модели SQL_DS_100M_ft с предметно-ориентированным бенчмарком «Data Scientist БИ» улучшает генерацию SQL-запросов, повышая их точность, структурированность и адаптивность для решения аналитических задач. Сравним результаты стандартной и тонко настроенной модели, проанализируем их производительность по различным категориям запросов и покажем, как адаптация улучшает понимание и применение SQL для пользователей. Пример запроса на естественном языке:

Запрос: Какие продукты и их категории имели среднюю цену за последний год, отсортированные по убыванию средней цены?

Для сравнения представим результат генерации SQL-запросов стандартной моделью SQL_DS_100M и тонко настроенной моделью SQL_DS_100M_ft. На рисунке 2а изображён запрос, созданный базовой моделью SQL_DS_100M, без использования специализированных предметных данных, а на рисунке 2б показан запрос, сгенерированный после контролируемой тонкой настройки модели SQL_DS_100M_ft.

Основные наблюдения:

Без использования тонкой настройки (SQL_DS_100M):

- SQL-запрос представляет собой один монолитный блок, объединяющий фильтр, соединение и группировку данных.
- Такая структура запроса снижает читабельность, особенно для студентов и пользователей с начальным уровнем подготовки.
- Отсутствие разделения логики фильтра и соединения делает запрос менее адаптивным для изменения.

```
SELECT
    p.product_id,
    AVG(p.avg_price) AS
avg_price,
    c.category_name
FROM
    products p
JOIN
    categories c ON
p.category_id = c.category_id
WHERE
    p.most_recent_order_timestamp
>= DATEADD(month, -12,
CURRENT_TIMESTAMP)
GROUP BY
    p.product_id,
    c.category_name
ORDER BY
    avg_price DESC
```

а) Генерация SQL-скрипта без учета контролируемой тонкой настройки. Модель SQL_DS_100M. Диалект SQLite3

```
WITH recent_orders AS (
    SELECT
        product_id,
        SUM(quantity) AS total_quantity
    FROM
        orders
    WHERE
        order_date >= DATEADD(month, -6,
CURRENT_TIMESTAMP)
    GROUP BY
        product_id
)
SELECT
    p.product_id,
    p.product_name,
    ro.total_quantity * p.avg_price AS
total_revenue
FROM
    products p
JOIN
    recent_orders ro ON p.product_id =
ro.product_id
ORDER BY
    total_revenue DESC
LIMIT 10;
```

б) Генерация SQL-скрипта с учетом контролируемой тонкой настройки. Модель SQL_DS_100M_ft. Диалект SQLite3

Рис. 2. Сравнение синтаксиса SQL без учета (2а) и с учетом 2(б) тонкой настройки модели SQL_DS_100M_ft

После тонкой настройки (SQL_DS_100M_ft):

- Логика запроса разделена на несколько этапов, представленных в виде Common Table Expressions (CTE).
- Основной запрос группирует и сортирует данные, обеспечивая прозрачность выполнения.

Эта структурированность не только улучшает читабельность, но и позволяет пошагово изучать отдельные элементы запроса, что особенно полезно в образовательных целях. Кроме того, подобный подход упрощает внесение изменений и адаптацию под новые задачи, так как каждое CTE можно модифицировать или использовать повторно.

Выводы. В данной работе представлен адаптивный подход к тонкой настройке языковых моделей для задач преобразования текста в SQL. Основное внимание уделено созданию специализированной SLM SQL_DS_100M_ft, которая демонстрирует значительное улучшение точности и адаптивности за счёт интеграции специализированных данных и методов обучения. Этот подход позволяет преодолеть ограничения крупных языковых моделей, такие как высокая стоимость вычислительных ресурсов и риск потери конфиденциальных данных.

Одним из ключевых аспектов является использование предметно-ориентированного бенчмарка «Data Scientist БИ», который обогащает обучающую выборку реальными схемами баз данных, процедурами, триггерами и функциями. Это позволяет адаптировать модель под специфические задачи аналитики, обеспечивая её гибкость и точность. В результате тонкая настройка модели SQL_DS_100M_ft приводит к улучшению качества прогнозируемых SQL-запросов, особенно для сложных категорий запросов, таких как объединения (join) и сортировка (order_by).

Основные преимущества предложенного подхода:

Высокая точность выполнения: Модель SQL_DS_100M_ft достигает точности 72.2% по метрике EX на сложных тестах (например, BIRD), демонстрируя конкурентоспособность с крупными языковыми моделями при значительно меньших вычислительных затратах.

Адаптивность к задачам аналитики: Использование специализированных данных (бенчмарк «Data Scientist БИ») повышает точность модели в категориях запросов (join, order_by).

Структурированность запросов: Разделение запросов на логические этапы (через CTE) улучшает их читабельность и делает их более подходящими для обучения и модификации.

Эффективность использования ресурсов: По сравнению с крупными моделями, такими как SQLCoder-7B-2, SQL_DS_100M_ft требует значительно меньших вычислительных ресурсов, что делает её доступной для образовательных и практических целей.

СПИСОК ИСТОЧНИКОВ

1. Haoyang, Li [et al] CodeS: Towards Building Open-source Language Models for Text-to-SQL // Proceedings of the ACM on Management of Data. – 2024. – V. 2, № 3. – 28 p. – DOI <https://doi.org/10.1145/3654930>.
2. Демонстрационная версия преобразования текста в SQL на базе CodeS [Электронный ресурс]. – URL: <https://github.com/RUCKBReasoning/text2sql-demo> (дата обращения: 01.02.2024).
3. Raymond, Li [et al] StarCoder: May the source be with you! [Электронный ресурс]. – 2023. – CoRR abs/2305.06161. – URL: <https://arxiv.org/abs/2305.06161> (дата обращения: 12.03.2024).
4. Tao, Yu [et al] Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task [Электронный ресурс]. – 2019. – URL: <https://arxiv.org/abs/1809.08887> (дата обращения: 12.10.2024).
5. Jinyang, Li [et al] Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs [Электронный ресурс]. – 2023. – URL: <https://arxiv.org/abs/2305.03111> (дата обращения: 01.10.2024).
6. Vaswani, A. [et al] Attention is All You Need // Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017. – 2017, Long Beach, CA, USA. – 2017. – P. 5998–6008.
7. Radford, A. [et al] Improving language understanding by generative pre-training [Электронный ресурс]. – 2018. – URL: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf (дата обращения: 10.11.2024).
8. Touvron, H. [et al] LLaMA: Open and Efficient Foundation Language Models // CoRR abs/2302.13971. – 2023. – URL: <https://arxiv.org/abs/2302.13971> (дата обращения: 12.10.2024).

9. Yin, P. [et al] TaBERT: Pretraining for Joint Understanding of Textual and Tabular Data // Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. – 2020. – P. 8413–8426.
10. Raffel, C. [et al] Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer // J. Mach. Learn. Res. – 2020. – V. 21. – P. 140:1–140:67.
11. Wei, J. [et al] Chain-of-Thought Prompting Elicits Reasoning in Large Language Models [Электронный ресурс] // NeurIPS. – 2022. – URL: <https://arxiv.org/abs/2201.11903> (дата обращения: 15.06.2024).
12. Yu, T. [et al] Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task [Электронный ресурс]. – 2019. – URL: <https://arxiv.org/abs/1809.08887> (дата обращения: 15.06.2024).
13. Zhong, V. [et al] Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning [Электронный ресурс]. – 2017. – URL: <https://arxiv.org/abs/1709.00103> (дата обращения: 12.06.2024).
14. Yu, T. [et al] CoSQL: A Conversational Text-to-SQL Challenge Towards Cross-Domain Natural Language Interfaces to Databases [Электронный ресурс]. – 2019. – URL: <https://arxiv.org/abs/1909.05378> (дата обращения: 15.06.2024).
15. Yu, T. [et al] SPaRC: Cross-Domain Semantic Parsing in Context [Электронный ресурс]. – 2019. – URL: <https://arxiv.org/abs/1906.02285> (дата обращения: 01.12.2023).
16. Li, J. [et al] Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs // CoRR. – 2023. – Vol. abs/2305.03111. – URL: <https://arxiv.org/abs/2305.03111> (дата обращения: 11.09.2024).
17. Модель SQLCoder-7B-2. Большая языковая модель для генерации SQL-кода на естественном языке [Электронный ресурс]. – URL: <https://huggingface.co/defog/sqlcoder-7b-2> (дата обращения: 11.09.2024).
18. Леонхардт, Х. Text2SQL (Revision c06880d) [Электронный ресурс]. – 2024. – URL: <https://huggingface.co/juanfra218/text2sql> (дата обращения: 12.10.2024). – DOI 10.57967/hf/2897.
19. Модель WPAIGPT-sql-01. Большая языковая модель для генерации SQL-кода на естественном языке [Электронный ресурс]. – URL: <https://huggingface.co/WPAI-INC/WPAIGPT-sql-01> (дата обращения: 10.09.2024).

Поступила в редакцию 23.12.2024 г., рекомендована к печати 20.01.2025 г.

ADAPTIVE APPROACH TO FINE-TUNING LANGUAGE MODELS FOR TEXT TO SQL CONVERSION TASKS

Bosenko T.M.

The ability to generate SQL queries based on natural language greatly simplifies data access for non-technical users. Current methods often rely on closed-source large language models (LLMs), such as ChatGPT, which have limitations including high cost and opaque architecture. In this paper, we propose an adaptive approach to fine-tuning small language models (SLMs) for converting text to SQL queries (Text-to-SQL) adapted to the SQLite3 and PostgreSQL dialects. The proposed model, SQL_DS_100M_ft, is based on the open-source CodeS architecture, which ensures its flexibility and accessibility. The analysis showed that fine-tuning can significantly improve the accuracy of SQL query generation (up to 72.2% by the EX-metric on BIRD tests) and the adaptability of the model for complex analytical tasks. The main advantages of such models are the structured nature of the generated queries, reduced costs for computing resources compared to LLM, and open up new prospects for improving the accessibility of relational databases through natural language interfaces.

Keywords: Text-to-SQL, language model, SQL queries, controlled fine-tuning, CodeS processing.

Босенко Тимур Муртазович

кандидат технических наук, доцент департамента информатики, управления и технологий ГАОУ ВО «Московский городской педагогический университет»,
Российская Федерация, г. Москва.
E-mail: bosenkotm@mgpu.ru

Bosenko Timur Murtazovich

Candidate of Technical Sciences, Associate Professor at Department of Computer Science, Management and Technology of Moscow City University,
Russian Federation, Moscow.